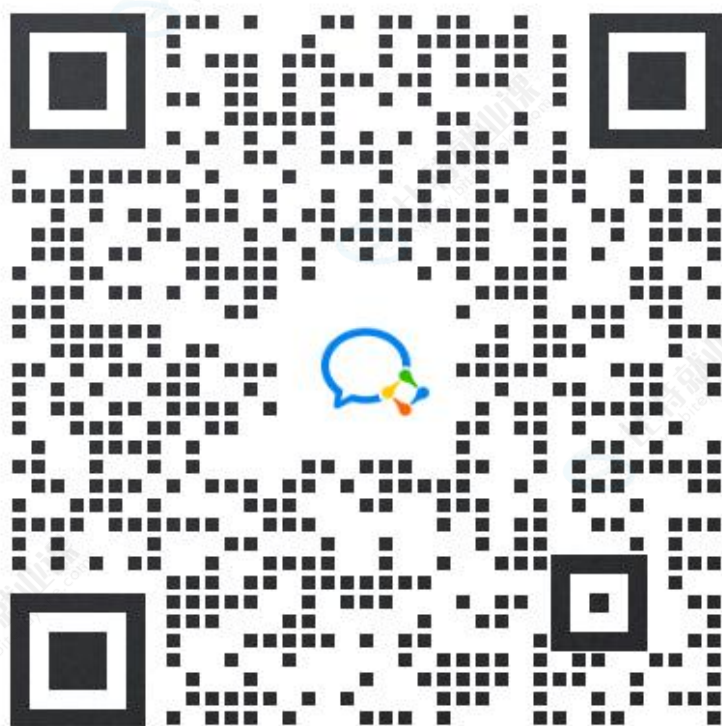


00-Vue3阶段必会的前置知识

版权说明

本 "比特就业课" Vue3全家桶核心基础及实战课程（以下简称“本项目”）的所有内容，包括但不限于文字、图片、音频、视频、软件、程序、数据库、设计、布局、界面等，均由本项目的开发者或授权方拥有版权。我们鼓励个人学习者使用本项目进行学习和研究。在遵守相关法律法规的前提下，个人学习者可以下载、浏览、学习本项目的内容，并为了个人学习、研究或教学目的而使用其中的材料。但请注意，**未经我们明确授权，个人学习者不得将本项目的内容用于任何商业目的**，包括但不限于销售、转让、许可或以其他方式从中获利。此外，个人学习者也不得擅自修改、复制、传播、展示、表演或制作本项目内容的衍生作品。任何未经授权的使用均属侵权行为，我们将依法追究法律责任。如果您希望以其他方式使用本项目的内容，包括但不限于引用、转载、摘录、改编等，请事先与我们取得联系，获取书面授权。感谢您对“比特就业课”项目的关注与支持，我们将持续努力，为您提供更好的学习体验。特此说明。比特就业课版权所有方。

对比特 课程/项目 感兴趣，可以联系这个微信。



代码 & 板书链接

<https://gitee.com/sinerry1314/bitejiuyeke-vue3>

一、变量和常量

1. 变量

```
1 let name = 'Jack'
2 let age = 18
3
4 name = 'Bit'
5 age = 20
```

2. 常量

```
1 const PI = 3.14
2
3 const articleList = []
4
5 const user = {
6   name: 'vue3',
7   age: 9
8 }
```

3. 思考

- const 声明的数组可以添加或删除么？
- const 声明的对象可以添加或修改属性么？

答：可以的，因为数组和对象在JS中属于引用类型，对其做添加、删除等操作，并不改变其内存地址。

```
1 const arr = [1,2,3]
2 // 添加
3 arr.push(4)
4 console.log(arr)
5 // 删除
6 arr.shift()
7 console.log(arr)
8 const obj = {
9   name: 'vue3',
10  age: 9
11 }
12
13 // 添加属性
14 obj.birth = 2015
15 console.log(obj)
16 // 修改属性
17 obj.age = 10
18 console.log(obj)
```

二、模版字符串

1. 普通字符串

用一对单引号或双引号声明, 基本都用单引号

```
1 // 普通字符串
2 let name = 'Jack'
3 let msg = "Hello World"
```

2. 模版字符串

用一对反引号声明

```
1 let name = `Jack`
2 let msg = `Hello World`
```

3. 好处

3.1 可任意换行

```
1 const htmlStr = '<div class="hot-goods-box"><h1>热门商品</h1><p>卖爆了卖爆了卖爆了</p></div>'
2
3 const htmlStr = `<div class="hot-goods-box">
4   <h1>热门商品</h1>
5   <p>卖爆了卖爆了卖爆了</p>
6 </div>`
```

3.2 可嵌入表达式

避免了繁琐的 加号 做字符串拼接; 嵌入的语法 `${表达式}`

```
1 let name = 'Bit'
2 let age = 9
3
4 // 把 name, age 变量的值放到 xxx 的位置, 并且展示是否成年
5 // let str = 'My name is xxx, I am xxx years old, 未成年 or 已成年'
6 // 传统的加号拼接
```

```
7 // let str = 'My name is ' + name + ', I am ' + age + ' years old' + (age >=
  18 ? '已成年' : '未成年')
8 // 模版字符串嵌入表达式
9 let str = `My name is ${name}, I am ${age} years old, ${age >= 18 ? '已成年' :
  '未成年'}`
```

三、对象

1. 取值

1.1 点取值

```
1 const obj = {
2   name: 'vue3',
3   age: 9
4 }
5
6 console.log(obj.name)
7 console.log(obj.age)
8
9 // 等同于
10 console.log(obj['name'])
11 console.log(obj['age'])
```

1.2 中括号取值

```
1 const obj = {
2   name: 'vue3',
3   age: 9
4 }
5
6 let x = 'name'
7 let y = 'age'
8
9 // 正确的
10 console.log(obj[x])
11 console.log(obj[y])
12
13 // 错误的
14 console.log(obj.x)
15 console.log(obj.y)
```

1.3 注意

当属性名是变量的时候, 只能用中括号取值; 否则既可以用点, 也可以用中括号

2. 简写

2.1 属性

当属性名和属性值的名字一样时, 并且配合变量时, 可以简写

```
1  let min = 1
2  let max = 99
3
4  // 以前的写法(不简写)
5  const obj = {
6    min: min,
7    max: max
8  }
9
10 // 现在的写法
11 const obj = {
12   min,
13   max
14 }
```

2.2 方法

```
1  const obj = {
2    // 不简写
3    fn: function() {
4
5    }
6  }
7
8  // 等同于
9
10 const obj = {
11   // 简写: 连同 :和function 一起省略
12   fn() {
13
14   }
15 }
```

四、解构赋值

1. 针对的目标

数组 或 对象

2. 作用

让数组和对象的取值更便捷

3. 代码示例

3.1 数组解构

```
1  const arr = [11, 22, 33]
2
3  // eg1: 把 arr 中的3个值分别赋值给变量 a, b, c
4
5  // 以前的写法
6  let a = arr[0]
7  let b = arr[1]
8  let c = arr[2]
9
10 // 现在的写法
11 let [a, b, c] = arr
12
13 // eg2: 把 arr 中的后两个值赋值给变量 b, c
14 let [, b, c] = arr
15
16 // eg3: 把 arr 中的第1个值赋值给变量 a, 剩余的全部给到变量 rest
17 let [a, ...rest] = arr
```

3.2 对象解构

```
1  const obj = {
2    name: '比特教育科技',
3    age: 9,
4    address: '陕西省西安市高新区沣惠南路34号'
5  }
6
7  // eg1: 把 obj 中的3个属性值分别赋值给变量 name, age, address
8
9  // 以前的写法
```

```

10  const name = obj.name
11  const age = obj.age
12  const address = obj.address
13
14  // 现在的写法
15  const { name, age, address } = obj
16
17  // eg2: 把 obj 的 age, address 属性值赋值给 age, address
18  const { age, address } = obj
19
20  // eg3: 把 obj 的 name 属性值赋值给变量 name, 剩余的全部给到变量 rest
21  const { name, ...rest } = obj
22
23  // eg4: 把 obj 的 name 属性值赋值给变量 uname
24  const { name: uname } = obj

```

4. 练习

```

1  const arr = [2,[3, 4],5]
2
3  // 1、把 arr 中的3, 4赋值给变量 a,b
4
5  const obj = {
6    data: {
7      code: 10000,
8      message: '频道列表获取成功',
9      result: ['HTML', 'CSS', 'JavaScript', 'Vue', 'SprintBoot']
10   },
11   status: 200,
12   statusText: 'Ok'
13 }
14
15 // 2、把 obj 中的 code, message, result 的值取出来赋值给变量 code, message, list

```

五、箭头函数

1. 非箭头函数

```

1  // 有名函数
2  function fn() {
3    // some code...
4  }

```

```
5 // 函数表达式
6 const fn = function() {
7     // some code...
8 }
```

2. 语法

```
1 const fn = () => {
2     // some code...
3 }
4
5 const add = (x, y) => {
6     return x + y
7 }
```

3. 特点

- 参数一个可以小括号、函数体一句话可以省略大括号

```
1 const log = arg => {
2     console.log(arg)
3 }
```

- 当省略了大括号时, 函数自带 return

```
1 const add = (x, y) => x + y
2
3 // 等同于
4 const add = (x, y) => {
5     return x + y
6 }
```

- 当函数体直接返回一个对象, 如果简写, 需要给对象加一对小括号

```
1 const state = () => ({
2     token: 'xxxx',
3     userInfo: {
4         name: 'admin',
5         id: 1
6     }
7 })
```



```
6    }
7  })
8
9  // 等同于
10 const state = () => {
11   return {
12     token: 'xxxx',
13     userInfo: {
14       name: 'admin',
15       id: 1
16     }
17   }
18 }
```

4. 应用

可用于普通函数的声明, 也多用于回调函数传参

```
1  setTimeout(() => {
2
3  }, 2000)
```

六、数组的重要方法

1. 概述

数组是js的重要数据结构, 掌握对数组的操作显得格外重要

2. 添加

push()和unshift()

```
1  const arr = [11, 22, 33]
2
3  // 尾部添加
4  const len = arr.push(44)
5  console.log(len) // 4
6
7  // 头部添加
8  const len = arr.unshift(44)
9  console.log(len) // 4
```

3. 删除

pop()和shift()

```
1  const arr = [11, 22, 33]
2
3  // 尾部删除
4  const last = arr.pop()
5  console.log(last) // 33
6
7  // 头部删除
8  const first = arr.shift()
9  console.log(first) // 11
```

4. 任意位置删除或添加

splice()

```
1  const arr = [11, 22, 33, 44]
2
3
4  // 语法
5  // arr.splice(startIndex:number, delCount:number, ...addItem)
6
7  // 删除 22
8  const temp = arr.splice(1, 1)
9  console.log(temp) // [22]
10 console.log(arr) // [11, 33, 44]
11
12 // 33后面添加55
13 arr.splice(2, 0, 55)
14 console.log(arr) // [11, 22, 33, 55, 44]
```

5. 包含

includes()

```
1  const arr = [11, 22, 33, 44]
2
3  console.log(arr.includes(33)) // true
4
5  console.log(arr.includes(55)) // false
```

6. 遍历

forEach()

```
1  const arr = [11, 22, 33, 44]
2
3  // for循环遍历
4  for(let i = 0; i < arr.length; i++) {
5      console.log(arr[i])
6  }
7
8  // forEach遍历
9  arr.forEach((item, index, array) => {
10     // item: 每次遍历的元素
11     // index: 元素的下标
12     // array: 数组本身
13 })
```

7. 过滤

filter()

保留满足条件的、去掉不满足条件的

```
1  const arr = [11, 22, 33, 44]
2
3  // 语法
4  const filteredArr = arr.filter((item, index, array) => {
5      // 内部会遍历数组，没遍历一次都会执行回调一次
6      // 如果返回true,则当前元素会保留；否则去掉
7      return 布尔值
8  })
9
10 // 保留所有的偶数
11 const eventArr = arr.filter((item) => {
12     if(item % 2 === 0) {
13         return true
14     } else {
15         return false
16     }
17 })
18
19 // 简写
20 const eventArr = arr.filter((item) => item % 2 === 0)
```

8. 映射

map()

由一个数组得到另一个数组, 并且二者长度相同、每个元素存在一一对应关系

```
1  const arr = [11, 22, 33, 44]
2  // 得到 [22, 44, 66, 88]
3
4  // 语法
5  const mappedArr = arr.map((item, index, array) => {
6    return 新值
7  })
8
9  // 得到每个元素翻倍的新数组
10 const doubleArr = arr.map((item) => {
11   // 原来每个元素乘 2
12   return item * 2
13 })
14
15 // 简写
16 const doubleArr = arr.map((item) => item * 2)
17
18
19 // 思考如何由arr得到如下数组
20 [
21   { index: 0, value: 11 },
22   { index: 1, value: 22 },
23   { index: 2, value: 33 },
24   { index: 3, value: 44 }
25 ]
26
27
28 const newArr = arr.map((value, index) => ({ index, value })))
```

9. 检测每一个

every()

```
1  const arr = [11, 22, 33, 44]
2
3  // 语法
4  arr.every((item, index, array) => {
```

```

5    // 1、如果返回true，说明当前元素满足条件，则继续检测下一次；
6    //    若都满足条件，则最终返回true
7    // 2、如果返回false，说明当前元素不满足条件；
8    //    立即停止检测，最终返回false
9    return 布尔值
10 })
11
12 // 判断arr中的元素是否都是奇数
13 const bool = arr.every((item) => {
14     console.log(item)
15     return item % 2 === 1
16 })
17
18 // 判断arr中的元素是否都大于10
19 const bool = arr.every((item) => {
20     console.log(item)
21     return item > 10
22 })

```

10. 汇总

reduce()

```

1    const arr = [11, 22, 33, 44]
2
3    // 对arr求和
4    let sum = 0
5    arr.forEach((item) => {
6        sum += item
7    })
8    console.log(sum)
9
10   // 语法
11   const result = arr.reduce((prev, item, index, array) => {
12       return 结果
13   }, 初始值)
14
15   // 对arr求和
16   const sum = arr.reduce((prev, item) => {
17       return prev + item
18   }, 0)
19
20   // 简写
21   const sum = arr.reduce((prev, item) => prev + item, 0)
22

```

```

23
24 // 商品列表数组
25 const goodsList = [
26   { id: 1, name: '篮球', num: 1 },
27   { id: 2, name: '玩具', num: 3 },
28   { id: 3, name: '书籍', num: 2 }
29 ]
30 // 求总数量?
31 const totalNum = goodsList.reduce((prev, item) => prev + item.num, 0)

```

七、对象的重要方法

Object.keys()

```

1  const obj = {
2    id: 10001,
3    name: 'Bit',
4    age: 9,
5    address: '陕西省西安市高新区沣惠南路34号'
6  }
7  // 之前遍历对象: for-in循环
8  for(let key in obj) {
9    console.log(key, obj[key])
10 }
11
12 // 获取对象键的数组
13 const keys = Object.keys(obj)
14 console.log(keys) // ['id', 'name', 'age']
15
16 // 可用来遍历对象
17 Object.keys(obj).forEach(key => {
18   console.log(obj[key])
19 })
20
21 // Object.keys()拿到的是键的数组, 可以对数组做很多处理在进行遍历对象
22
23 // 需求: 获取 obj 所有以 a 开头的属性值
24 Object
25   .keys(obj)
26   .filter(key => key.startsWith('a'))
27   .forEach(key => {
28     console.log(key, obj[key])
29   })

```

八、扩展运算符

...

1. 复制数组或对象

```
1  const arr1 = [11, 22, 33]
2  // 赋值
3  const arr2 = arr1
4  arr2.push(44)
5  console.log(arr1) // 受影响了
6
7  // 正确的做法, 把 arr1 复制一份给到 arr2
8  const arr2 = [...arr1]
9
10
11 const obj1 = {
12   id: 10001,
13   name: 'Bit',
14   age: 9
15 }
16 // 赋值
17 const obj2 = obj1
18 obj2.age = 10
19 console.log(obj1) // 受影响了
20
21 // 正确的做法, 把 obj1 复制一份给到 obj2
22 const obj2 = {...obj1}
```

2. 合并数组或对象

```
1  const arr1 = [1,2,3]
2  const arr2 = [4,5,6]
3  // 把 arr1 和 arr2 合并起来给到 arr
4  const arr = [...arr1, ...arr2]
5
6
7  const obj1 = {
8   name: 'Jack',
9   height: 176
10 }
11 const obj2 = {
12   height: 180,
13   age: 18
```

```
14 }
15 // 把 obj1 和 obj2 合并起来给到 obj
16
17 // 注意：同名属性会覆盖
18 const obj = {
19   ...obj1,
20   ...obj2
21 }
```

九、序列化和反序列化

1. 序列化

把对象转换为 json 格式字符串

```
1 // 对象
2 const json = {
3   id: 10001,
4   name: 'Bit',
5   age: 9
6 }
7 // 序列化
8 const jsonStr = JSON.stringify(json)
```

2. 反序列化

把 json 字符串转换为 json 数据

```
1 // json字符串
2 const jsonStr = '{"id": 10001, "name": "Bit", "age": 9}'
3
4 // 反序列化
5 const json = JSON.parse(jsonStr)
```

十、Web存储

1. 介绍

Web Storage 包含如下两种机制：

- `sessionStorage` 该存储区域在页面会话期间可用（即只要浏览器处于打开状态，包括页面重新加载和恢复）。

- 仅为会话存储数据，这意味着数据将一直存储到浏览器（或选项卡）关闭。
- 数据永远不会被传输到服务器。
- 存储限额大于 Cookie（最大 5MB）。
- `localStorage` 即使浏览器关闭并重新打开也仍然存在。
 - 存储的数据没有过期日期，只能通过 JavaScript、清除浏览器缓存或本地存储的数据来清除。

2. 用法

以 `localStorage` 为例, 学习存、取、删

2.1 存

```
1 // 存
2 localStorage.setItem(key:string, value:string)
3
4 // eg:
5 localStorage.setItem('uname', 'Bit')
6
7 // 存对象和数组, 需要序列化
```

2.2 取

如果 `key` 存在, 则取出相应的值; 否则取出的值为 `null`

```
1 // 取
2 localStorage.getItem(key:string)
3
4 // eg:
5 const uname = localStorage.getItem('uname')
6
7 // 取出对象和数组, 需要反序列化
```

2.3 删

```
1 // 删
2 localStorage.removeItem(key:string)
3
4 localStorage.removeItem('uname')
```

2.4 注意

存储对象和数组需要进行序列化和反序列化

```
1  const obj = {
2    id: 10001,
3    name: 'Bit',
4    age: 9
5  }
6
7  // 存: 序列化
8  localStorage.setItem('bit', JSON.stringify(obj))
9  // 取: 反序列化
10 const local = JSON.parse(localStorage.getItem('bit'))
```

十一、Promise+Async/Await

1. 为什么需要Promise

是为了消除回调地狱的

```
1  // 需求: 延迟2秒之后输出1, 完了之后延迟1秒输出2, 完了之后延迟1秒输出3
2  setTimeout(() => {
3    console.log(1)
4    setTimeout(() => {
5      console.log(2)
6      setTimeout(() => {
7        console.log(3)
8      }, 1000)
9    }, 1000)
10 }, 2000)
11
12 // 上述代码存在的问题: 回调套回调, 代码的可读性差
```

为了解决上述问题、Promise就应运而生了

2. Promise介绍

Promise是一个类, 用来包装异步操作, 根据异步操作的结果, 是成功还是失败, 进而决定Promise是成功还是失败; Promise支持链式调用, 从而消除回调地狱

3. Promise的3种状态

- 1 **Pending**: 进行中
- 2 **Fulfilled**: 成功
- 3 **Rejected**: 失败

Promise的状态

1. 只能由 **Pending -> Fulfilled**, 或 **Pending -> Rejected**
2. Promise的状态一旦确定, 就不可改变了

4. 基本使用

```
1  const p = new Promise((resolve, reject) => {
2    // 这里编写异步代码: 比如定时器、ajax请求等
3    setTimeout(() => {
4      // 2秒后, Promise标记为成功
5      resolve('ok')
6
7      reject('error')
8    }, 2000)
9  })
10
11  p.then((msg) => {
12    // 成功回调
13    console.log(msg) // ok
14  }, (err) => {
15    // 失败回调
16    console.log(err) // error
17  })
```

5. 消除上述回调地狱

```
1  // 封装延迟函数
2  function delay(duration, n) {
3    return new Promise((resolve) => {
4      setTimeout(() => {
5        resolve(n)
6      }, duration)
7    })
8  }
```

```

9
10 // 链式调用消除回调地狱
11 delay(2000, 1)
12   .then(n1 => {
13     console.log(n1)
14     return delay(1000, 2)
15   }).then(n2 => {
16     console.log(n2)
17     return delay(1000, 3)
18   }).then(n3 => {
19     console.log(n3)
20   })

```

上述代码虽然消除了回调地狱、但链式调用过长、也不利于阅读
为了继续优化、Async+Await就应运而生了

6. Async+Await异步终极解决方案

```

1 // 封装延迟函数
2 function delay(duration, n) {
3   return new Promise((resolve) => {
4     setTimeout(() => {
5       resolve(n)
6     }, duration)
7   })
8
9
10 // 语法
11 async function log() {
12   // 1、在Promise实例前添加 await 关键字，那么await的返回值就是当前
13   // Promise的resolve参数
14   // 2、await所在的函数必须被async修饰
15   // 3、async函数内，当前await执行结束了，代码才会继续往后执行(同步的方式执行)
16   const n1 = await delay(2000, 1)
17   console.log(n1)
18 }
19 log()
20
21
22 // 最终代码
23 async function log() {
24   const n1 = await delay(2000, 1)
25   console.log(n1)
26   const n2 = await delay(1000, 2)

```

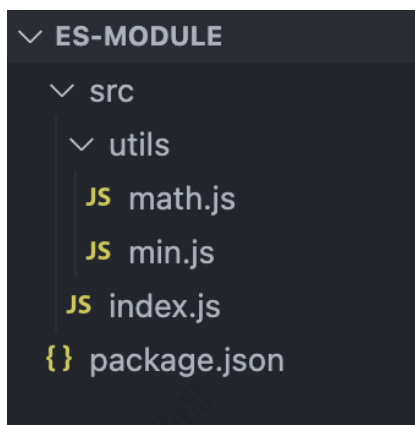
```
27     console.log(n2)
28     const n3 = await delay(1000, 3)
29     console.log(n3)
30 }
31
32 log()
```

十二、模块化

1. 概述

模块化是指将一个复杂程序划分为一系列独立、可互操作的模块的过程。每个模块负责特定的功能或任务，并通过定义好的接口与其他模块进行通信。简单来说，就是将代码进行分解、按功能进行管理。模块化的目的是提高代码的可维护性、可重用性、可测试性和可扩展性。开发者能够更容易地处理大型JavaScript项目。

2. 目录结构准备



1. 新建 12-es-module 目录
2. 命令行进入 12-es-module 目录
3. 执行 `npm init` ,初始化得到 `package.json` 文件
4. 给 `package.json` 添加 `"type": "module"`
5. 根目录下新建 `src/index.js` 、作为代码的入口文件
6. `src` 目录下新建 `utils` 目录

3. 默认导出与导入

新建 `utils/min.js`

```
1 // 定义并默认导出求最小值函数
2 export default function min(m, n) {
```

```
3     return m > n ? n : m
4 }
5
6 // 注意：默认导出 export default 在一个模块中最多出现1次
```

src/index.js

```
1 // 默认导入
2 import min from './utils/min.js'
3
4 console.log(min(12, 45))
```

4. 按需导出与导入

新建 utils/math.js

```
1 // 定义求和函数并按需导出
2 export function add(x, y) {
3     return x + y
4 }
5
6 // 定义作差函数并按需导出
7 export function sub(x, y) {
8     return x - y
9 }
10
11 // 注意：按需导出 export 在一个模块中可以出现多次
```

src/index.js

```
1 // 按需导入
2 import { add, sub } from './utils/math.js'
3
4 console.log(add(33, 18))
5 console.log(sub(33, 18))
```